LECTURES

00 Destructuring | 15 Navigator, History | 23 Promises | 28 AJAX | ★ Cheat sheet

REVISION NOTES · JAVASCRIPT

JavaScript Promises and AJAX Notes

Condensed from four decks: Objects destructuring, History and Navigator object, Promise and AJAX. Code is cleaned up from the slides (curly quotes and typos fixed). Points marked "Extra" go slightly beyond the slides.

Destructuring

Deck 00 · objects_destructing

Destructuring (ES6) breaks an object or array into smaller parts and puts them straight into variables. Without it you repeat code like this:

```
let options = { repeat: true, save: false };
let repeat = options.repeat,
    save = options.save;          // one line per value
```

The pattern on the left uses the same syntax as an object or array literal.

Object destructuring

```
let node = { type: "Identifier", name: "foo" };
let { type, name } = node;        // variables named after the properties
console.log(type);                // "Identifier"
console.log(name);                // "foo"
```

DON'T FORGET THE INITIALIZER

With `var`, `let` or `const`, a destructuring declaration needs the `= value` part. `let { type, name };` is a SyntaxError. (A plain `let x;` is fine; only `const` always needs an initializer.)

Destructuring assignment (variables already exist)

```
let node = { type: "Identifier", name: "foo" },
    type = "Literal",
```

```
    name = 5;

({ type, name } = node);           // parentheses are required
console.log(type);                 // "Identifier"
console.log(name);                 // "foo"
```

- The brackets are needed because a `{` at the start of a statement is read as a block, and a block cannot be on the left of `=`.
- The assignment expression evaluates to the **right side**, so it can be used where a value is expected: `outputInfo({ type, name } = node)` passes `node` to the function.
- If the right side is `null` or `undefined`, an error is thrown, because reading a property of them fails.

In a function parameter

```
function outputInfo({ type, name }){
  console.log(type);    // "Identifier"
  console.log(name);    // "foo"
}
outputInfo(node);       // outer variables type = "Literal" and name = 5 stay unchange
```

Default values

A name that does not exist as a property gives `undefined`. Add `= value` for a default, used when the property is missing or is `undefined`.

```
let { type, name, value } = node;
console.log(value);                // undefined

let { type, name, value = true } = node;
console.log(value);                // true
```

Using a different variable name

```
let { type: localType, name: localName } = node;
console.log(localType);            // "Identifier"
console.log(localName);            // "foo"
console.log(type);                 // error: no variable named type was created

let { type: localType, name: localName = "bar" } = { type: "Identifier" };
console.log(localName);            // "bar"   default goes after the local name
```

READING THE COLON

READING THE COLON

In `{ type: localType }` the property to

READ

is on the left of the colon and the variable to

CREATE

is on the right. This is the opposite of an object literal, where the name is on the left and the value on the right.

Nested objects

```
let node = {
  type: "Identifier", name: "foo",
  loc: { start: { line: 1, column: 1 }, end: { line: 1, column: 4 } }
};
let { loc: { start } } = node;                 // start = { line: 1, column: 1 }
start.line;                                    // 1

let { loc: { start: localStart } } = node;     // rename while descending
let { loc: { start: { line } } } = node;       // go deeper: line = 1
```

A colon followed by `{` means "go one level deeper". The pattern can nest to any depth.

Array destructuring

Works by **position**, not by name, so the variable names can be anything. The original array is not changed.

```
let colors = ["red", "green", "blue"];
let [firstColor, secondColor] = colors;    // "red", "green"
let [ , , thirdColor] = colors;            // "blue"   commas skip positions

let a = "black", b = "purple";
[a, b] = colors;                           // assignment: NO parentheses needed
// a is "red", b is "green"
```

Swapping two variables

```
// ES5: needs a temporary variable
let a = 1, b = 2, tmp;
tmp = a; a = b; b = tmp;

// ES6
let a = 1, b = 2;
[a, b] = [b, a];       // a is 2, b is 1
```

The right side builds a temporary array `[b, a]`, and the left side unpacks it back into `a` and `b`.

Defaults, nesting, rest items

```
let [first, second = "green"] = ["red"];        // second is "green" (default used)

let colors = ["red", ["green", "lightgreen"], "blue"];
let [firstColor, [secondColor]] = colors;       // "red", "green"   extra [] to go int

let [head, ...restColors] = ["red", "green", "blue"];
head;                    // "red"
restColors;              // ["green", "blue"]   a new array
restColors.length;       // 2
```

- `...rest` collects the remaining items into an array. It must be the last element in the pattern (Extra).
- **Cloning an array:** ES5 used `colors.concat()` (with no argument it returns a copy). ES6 can use `let [...clonedColors] = colors;`. The concat method with an argument joins arrays: `colors.concat(colors2)`.
- Extra: both ways give a shallow copy. Nested objects inside are still shared.

	OBJECT PATTERN `{ }`	ARRAY PATTERN `[ ]`
Matches by	Property name	Position
Rename	`{ type: localType }`	Just pick any name
Skip items	Leave the property out	Use empty commas
Assignment (no declaration)	Needs `( )` around it	No brackets needed
Default	`{ value = true }`	`[a, b = "green"]`

Navigator and History objects

Deck 15 · History_navigator_Object

navigator: information about the visitor's browser

```
var browser = navigator.appName;
var b_version = navigator.appVersion;
```

```
var apcode = navigator.appCodeName;
var lang = navigator.language;
var b = navigator.cookieEnabled;
document.write("Browser name: " + browser + "<br />");
document.write("Browser version: " + b_version + "<br />");
document.write("app_code: " + apcode + "<br />");
document.write("lang: " + lang + "<br />");
document.write("cookie enabled: " + b);
// Browser name: Netscape  Browser version: 5.0 (Windows)  app_code: Mozilla  lang: en
```

PROPERTY	RETURNS
`appName`	Browser application name. Oddly, it is "Netscape" for IE11, Chrome, Firefox and Safari
`appCodeName`	Code name. It is "Mozilla" for Chrome, Firefox, IE, Safari and Opera
`appVersion`	Version information
`language`	Browser language, e.g. `en-US`
`cookieEnabled`	`true` if cookies are enabled
`onLine`	`true` if the browser is online
`platform`	Browser platform (operating system)

WATCH OUT

Do not rely on `appName` or `appCodeName` to detect a browser, because every browser reports the same values. The deck mentions the `browscap.ini` file as a better source of browser capabilities.

history: move through visited pages

CALL	SAME AS
`history.back()`	Clicking the browser's Back button
`history.forward()`	Clicking the browser's Forward button
`history.go(n)` (Extra)	Jump n pages: `go(-2)` back two, `go(1)` forward one

```
<!-- first.html: run this first and click the link -->
<a href="copy.html">go to next file</a>
<input type="button" value="next" onclick="history.forward()">
```

```
<!-- copy.html -->
<script>
  function copy(){ document.getElementById("b").value = document.getElementById("a").v
</script>
<input type="text" id="a">
<input type="button" onclick="copy()" value="copy data">
<input id="b">
<input type="button" value="back" onclick="history.back()">
```

`forward()` only works after you have gone back, because there must be a next page in the history. That is why the deck says to open the first page and click the link before trying the button.

Promises

Deck 23 · promise

The idea: a subscription list

A singer promises fans a new song. Fans put their email on a list, and everyone on it gets the song when it is published. If something goes wrong (a fire in the studio), they are told that too.

ANALOGY	IN CODE
The singer	**Producing code**: does something that takes time, such as loading data over a network
The fans	**Consuming code**: functions that want the result once it is ready
The subscription list	The **Promise** object that links producer and consumers

Creating a promise

```
let promise = new Promise(function(resolve, reject){
  // executor: the producing code
});
```

- The function passed to `new Promise` is the **executor**. It runs automatically and immediately when the promise is created.
- `resolve` and `reject` are callbacks supplied by JavaScript. You do not define them;

you only call one of them.

- `resolve(value)` : the job succeeded, with result `value` .
- `reject(error)` : something went wrong, with error object `error` .

INTERNAL PROPERTY	STARTS AS	AFTER `RESOLVE(VALUE)`	AFTER `REJECT(ERROR)`
state	"pending"	"fulfilled"	"rejected"
result	undefined	value	error

A promise that is fulfilled or rejected is **settled**. `state` and `result` are internal and cannot be read directly. You use `.then` , `.catch` and `.finally` instead.

```
// success after 1 second
let promise = new Promise(function(resolve, reject){
  setTimeout(() => resolve("done"), 1000);
});

// failure after 1 second
let promise2 = new Promise(function(resolve, reject){
  setTimeout(() => reject(new Error("Whoops!")), 1000);
});
```

Rules for the executor

- **Only one result.** The first `resolve` or `reject` wins and any state change is final. Later calls are ignored.
- `resolve` and `reject` take only one argument (or none). Extra arguments are ignored.
- Reject with an **Error object** (or something that inherits from Error). It is recommended, and it helps with error handling.
- The executor may call `resolve` straight away, for example when the answer is already cached: `new Promise(resolve => resolve(123))` gives an immediately resolved promise.

```
new Promise(function(resolve, reject){
  resolve("done");
  reject(new Error("…"));          // ignored
  setTimeout(() => resolve("…"));  // ignored
});
```

Consumers: then, catch, finally

```
promise.then(
```

```
  function(result){ /* runs if resolved */ },
  function(error){  /* runs if rejected */ }
);

promise.then(result => alert(result), error => alert(error));   // "done!" or "Error:
promise.then(alert);                                            // only interested in s
```

METHOD	RUNS WHEN	NOTES
`.then(onResolve, onReject)`	Resolved (1st function) or rejected (2nd function)	The 2nd function is optional
`.catch(f)`	Rejected	Exactly `.then(null, f)`, just shorter
`.finally(f)`	Settled either way	Cleanup, like `finally` in try/catch. Gets **no arguments**

```
promise.catch(err => alert(err));            // same as promise.then(null, err => aler

new Promise((resolve, reject) => { /* slow work, then resolve/reject */ })
  .finally(() => stopLoadingIndicator())     // always runs first, whatever the outcom
  .then(result => show(result), err => show(err));
```

How finally differs from then(f, f)

- A `finally` handler has no arguments: it cannot tell whether the promise succeeded.
- It **passes the result or error through** to the next handler.

```
new Promise((resolve) => setTimeout(() => resolve("result"), 2000))
  .finally(() => alert("Promise ready"))
  .then(result => alert(result));            // "result" arrives here

new Promise(() => { throw new Error("error"); })   // throwing in the executor rejects
  .finally(() => alert("Promise ready"))
  .catch(err => alert(err));                 // the error arrives here
```

Handlers can be added after the promise settles

```
let promise = new Promise(resolve => resolve("done!"));
promise.then(alert);       // runs right away: "done!"
```

If the promise is still pending, handlers wait. If it has already settled, they run immediately.

This is more flexible than the singer analogy, where signing up after release would miss the song.

Reading a chain

```
new Promise((resolve, reject) => { throw new Error("error"); })
  .finally(() => alert("Promise ready"))      // 1. runs, passes the error on
  .then(r => alert(r), err => alert(err))     // 2. error handler runs, shows the erro
  .then(r => alert(r))                        // 3. receives undefined, because step 2
  .catch(err => alert(err));                  // never runs: step 2 already handled th
```

Promises vs callbacks

PROMISES	CALLBACKS
Natural order: run `loadScript()`, then use `.then` to say what to do with the result	You must already have the callback ready, so you must know what to do with the result before you start
Call `.then` as many times as you like; each adds another subscriber	Only one callback

```
// Callback
setTimeout(function(){ myFunction("I love JS !!!"); }, 3000);
function myFunction(value){ document.getElementById("demo").innerHTML = value; }

// Promise
let myPromise = new Promise(function(myResolve, myReject){
  setTimeout(function(){ myResolve("I love JS !!"); }, 3000);
});
myPromise.then(value => { document.getElementById("demo").innerHTML = value; });
```

Loading a file, callback version and promise version:

```
// Callback
function getFile(myCallback){
  let req = new XMLHttpRequest();
  req.open('GET', "mycar.html");
  req.onload = function(){
    if (req.status == 200) myCallback(this.responseText);
    else myCallback("Error: " + req.status);
  };
  req.send();
}
getFile(myDisplayer);
```

```
// Promise
let myPromise = new Promise(function(myResolve, myReject){
  let req = new XMLHttpRequest();
  req.open('GET', "mycar.html");
  req.onload = function(){
    if (req.status == 200) myResolve(req.response);
    else myReject("File not Found");
  };
  req.send();
});
myPromise.then(
  function(value){ myDisplayer(value); },
  function(error){ myDisplayer(error); }
);
```

In the promise version, success and failure are reported through `resolve` and `reject` and handled in two separate functions, so the loading code no longer has to know what the caller will do with the result.

AJAX

`Deck 28 · ajax`

What it is

- **A**synchronous **J**avaScript **A**nd **X**ML.
- Not a new language. It is a technique for faster, more interactive web pages, built on JavaScript, XML, HTML and CSS.
- JavaScript talks to the server through the `XMLHttpRequest` object and exchanges data **without reloading the page**.
- Only small pieces of data travel (usually JSON or XML), not whole pages. The user stays on the same page and does not notice the background request.
- It is a browser technology, independent of the server software.

TRADITIONAL FORM	AJAX
User clicks Submit, waits, and a new page loads with the result	Script sends a request in the background and updates just part of the page
Server returns a whole page each time, so it feels slower	Server returns only the data needed

How a request flows

1. An event happens on the page (it loads, a button is clicked).
2. JavaScript creates an `XMLHttpRequest` object.
3. The object sends a request to the web server.
4. The server processes the request.
5. The server sends a response back.
6. JavaScript reads the response.
7. JavaScript takes the right action, such as updating part of the page.

Old Internet Explorer used an `ActiveXObject`; every other browser uses the built-in `XMLHttpRequest`.

XMLHttpRequest methods

METHOD	DESCRIPTION
`new XMLHttpRequest()`	Creates the object
`open(method, url, async, user, psw)`	Sets up the request. `method` is GET or POST, `url` the file or script, `async` true (default) or false. It only **prepares** the request; it does not contact the server
`send()`	Sends the request (GET). This is when the connection happens and the download starts
`send(string)`	Sends the request with data in the body (POST)
`setRequestHeader()`	Adds a label/value pair to the headers. Needed for POST
`abort()`	Cancels the current request
`getAllResponseHeaders()`, `getResponseHeader()`	Return header information

XMLHttpRequest properties

PROPERTY	DESCRIPTION
`onreadystatechange`	Function called every time `readyState` changes
`readyState`	Progress of the request, 0 to 4 (below)

`responseText`	Response as a string
`responseXML`	Response as XML data
`status`	Number: 200 OK, 403 Forbidden, 404 Not Found
`statusText`	Text for the status, e.g. "OK" or "Not Found"

READYSTATE	MEANING
0	Request not initialised
1	Server connection established (request set up)
2	Request received (sent)
3	Processing request
4	Request finished and response is ready

THE STANDARD CHECK

Do your work only when `readyState == 4` (finished)
AND
`status == 200` (OK).

Basic GET example

```
<button type="button" onclick="loadDoc()">Request data</button>
<p id="demo"></p>
<script>
function loadDoc(){
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function(){
    if (this.readyState == 4 && this.status == 200){
      document.getElementById("demo").innerHTML = this.responseText;
    }
  };
  xhttp.open("GET", "ejson.json", true);
  xhttp.send();
}
</script>
```

The page must be loaded through a web server. Typing `location.protocol` in the console should show `http:` ; if it shows `file:` the request will usually be blocked.

Reading JSON

```
// ejson.json
[ { "code": "emp101", "name": "Tom",  "gender": "Male",   "annualSalary": "5500",    "
  { "code": "emp102", "name": "Alex", "gender": "Male",   "annualSalary": "5700.95", "

xhttp.onreadystatechange = function(){
  if (this.readyState == 4 && this.status == 200){
    let arr = JSON.parse(this.responseText);   // text -> array of objects
    let fd = "";
    for (let x of arr){
      for (let b in x) fd += x[b] + "<br>";   // each property value
      fd += "<hr>";
    }
    document.getElementById("demo").innerHTML = fd;
  }
};
```

- `responseText` is always a string. `JSON.parse()` turns it into real objects and arrays.
- The opposite is `JSON.stringify(obj)`, which turns an object into JSON text to send.
- `for...of` goes over the array elements; `for...in` goes over each object's property names.

GET or POST

Use POST when:

- A cached copy is not acceptable (you are updating a file or database on the server).
- You are sending a large amount of data (POST has no size limit).
- You are sending user input, which can contain unknown characters. POST is more robust and secure than GET.

Otherwise GET is fine for simple reads.

Sending form data with POST

POST puts the data in `send(...)` and needs a `Content-Type` header set after `open`.

```
function loadDoc(){
  var form = document.querySelector('form');
  var data = new URLSearchParams(new FormData(form)).toString();   // "fnm=Alice&ag=25

  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function(){
    if (this.readyState == 4 && this.status == 200)
      document.getElementById("demo").innerHTML = "Data inserted";
```

```
  };
  xhttp.open("POST", "ejson.php", true);
  xhttp.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded');
  xhttp.send(data);
}
```

- `new FormData(form)` collects all fields that have a `name` attribute and are not disabled.
- `new URLSearchParams(formData)` turns that into a URL-encoded set of pairs, and `.toString()` gives text such as `name=Alice&age=25`.
- You can loop over a FormData: `for (var [key, value] of data) { ... }`, or fill your own object: `jsondata[key] = value`, then `JSON.stringify(jsondata)`.
- On the server, PHP reads `$_POST['fnm']` and `$_POST['ag']`. In an ASP.NET REST API: `[HttpPost("submit")] Submit([FromForm] PersonForm form)`.

CORRECTION TO ONE SLIDE

One example builds a JSON string with `JSON.stringify(data)` but still sets the header to `application/x-www-form-urlencoded`. The header must match the body. For a JSON body use `application/json`.

Content-Type values

CONTENT-TYPE	WHAT IT SENDS	TYPICAL USE
`application/x-www-form-urlencoded`	URL-encoded key=value pairs (the default for forms)	Simple forms: login, search
`multipart/form-data`	Several parts, text and files	File uploads: images, documents
`application/json`	JSON objects and arrays	REST API calls, modern web apps

Reusable callback pattern

If a site has several AJAX tasks, write **one** function that does the `XMLHttpRequest` work and one callback function for each task. The call passes the URL and the function to run when the response is ready.

```
<button onclick="loadDoc('ajax_info.txt', myFunction)">get ajax</button>
<button onclick="loadDoc('read_data.txt', go)">read_data</button>
<div id="demo"></div><div id="dd"></div>
<script>
```

```
<script>
function loadDoc(url, cFunction){
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function(){
    if (this.readyState == 4 && this.status == 200) cFunction(this);   // hand the req
  };
  xhttp.open("GET", url, true);
  xhttp.send();
}
function myFunction(xhttp){ document.getElementById("demo").innerHTML = xhttp.response
function go(xhttp){         document.getElementById("dd").innerHTML = xhttp.responseTe
</script>
```

Connecting AJAX and promises

The Promise deck wraps the same `XMLHttpRequest` in a promise: `resolve` on status 200, `reject` otherwise. Extra: the modern built-in alternative is `fetch()`, which already returns a promise, so you write `fetch(url).then(r => r.json()).then(data => ...)`. The last slide of the deck points to a StackOverflow discussion of fetch versus AJAX for further reading.

Last-minute cheat sheet

`All four decks`

Likely exam questions

- What is destructuring; object vs array destructuring; renaming, defaults, nesting.
- Why object destructuring assignment needs parentheses; why a declaration needs an initializer.
- Swap two variables with destructuring; clone an array with rest items.
- Properties of `navigator`; why not to trust `appName`; `history.back()` and `forward()`.
- What a Promise is; its states (`pending`, `fulfilled`, `rejected`); what the executor does.
- `then`, `catch`, `finally`: arguments and differences.
- Promises vs callbacks, with a table or code.
- What AJAX is, its steps, and why the page does not reload.
- `readyState` values 0 to 4; the `readyState == 4 && status == 200` check.
- GET vs POST and when to use POST; the three Content-Type values.

- Write an AJAX call that reads a JSON file and shows it in the page.

Easy mistakes

- Writing `{ type, name } = node;` without parentheses.
- Reading the rename colon backwards: `{ type: localType }` creates `localType`, not `type`.
- Calling both `resolve` and `reject` and expecting both to work. Only the first counts.
- Expecting `finally` to receive the result. It gets no arguments.
- Putting side effects in the executor and forgetting it runs immediately.
- Forgetting that `open()` does not contact the server; `send()` does.
- Skipping the `readyState == 4` check and running the code several times.
- Using `responseText` as an object. Parse it first with `JSON.parse`.
- Forgetting `setRequestHeader` for POST, or using a header that does not match the body.
- Testing AJAX by opening the file directly (`file:`) instead of through a server.